A Context-Aware Agent Harness (LexiGuide)

Rupayan Chattaraj (24BCE0716), Nagendra (24BC2185), Himanshu (24BCE0955)
School of Computer Science and Engineering, Vellore Institute of Technology, Vellore

***Abstract* —** *Traditional compilers detect lexical errors but often provide technical and unclear messages that fail to explain the underlying cause or suggest an appropriate correction, leaving beginners struggling to resolve them efficiently. This review surveys the existing body of work on compiler error explanation and repair, spanning Large Language Model (LLM)-based explanation systems, Transformer-based program repair, and student-oriented error-correction frameworks. Building on the identified gaps, we propose LexiGuide, a context-aware system that combines lexical error detection, plain-language explanation, and context-sensitive correction suggestions within a single unified workflow. The proposed system aims to make compiler feedback more understandable and actionable, particularly for novice programmers.*

***Keywords* —** *lexical error detection, compiler diagnostics, large language models, context-aware program repair, error explanation, novice programmers*

I. INTRODUCTION

Compilers remain the primary interface through which programmers, especially students, learn to reason about the correctness of their code. However, the diagnostic messages produced by traditional compilers are frequently terse, technical, and disconnected from the broader context of the program in which the error occurs. This gap between what a compiler reports and what a programmer actually needs to understand is particularly pronounced for lexical errors, where the message often names the offending token without explaining why it is invalid or how it should be fixed.

This problem motivates the present work. We introduce LexiGuide, a context-aware system that automatically detects lexical errors, explains their cause in simple language, and generates context-sensitive correction suggestions, thereby making compiler feedback more understandable and actionable for beginners.

II. PROBLEM IDENTIFICATION

A. Identification Problems

1. Traditional compiler error messages are often difficult for beginners to understand.
2. Compiler messages usually identify the error but do not provide sufficient explanation about why the error occurred.
3. Programmers frequently need to search external platforms such as Stack Overflow to understand and resolve compiler errors.
4. Existing solutions may not always be specific to the programmer's particular code and context.
5. Converting a compiler error message into an actionable solution can be challenging for inexperienced programmers.
6. Automatically generated explanations may sometimes be inaccurate or misleading.

B. Proposed Solutions

1. Use Large Language Models (LLMs) to generate simple and understandable explanations of compiler errors.
2. Provide contextual information along with the error message to explain the cause of the error.
3. Reduce dependency on external sources by generating explanations and possible solutions directly from the error and source code.

4. Analyze the specific source code and compiler error together to provide context-specific suggestions.
5. Provide actionable correction suggestions along with explanations to help programmers resolve errors.
6. Validate generated suggestions using compiler diagnostics and program context to improve their reliability.

III. LITERATURE SURVEY

Table I summarizes twelve research papers that inform the design of LexiGuide, spanning LLM-based error explanation, Transformer-based program repair, and student-focused compilation-error correction. Each entry lists the paper's principal focus and its specific relevance to the proposed system.

TABLE I. SURVEYED RESEARCH PAPERS AND THEIR RELEVANCE TO LEXIGUIDE

No.	Research Paper	Year	Main Focus	Relevance to LexiGuide
1	Addressing Compiler Errors: Stack Overflow or Large Language Models? — P. Widjojo, C. Treude	2023	Compares Stack Overflow and LLMs for explaining compiler errors; GPT-4 outperformed in a study of 100 compiler error messages.	Supports the error-explanation component.
2	TransRepair: Context-aware Program Repair for Compilation Errors — X. Li et al.	2022	Uses erroneous code context and compiler diagnostic feedback with a Transformer to repair C programs.	Highly relevant to context-aware correction.
3	Compilation Error Repair: For the Student Programs, From the Student Programs — U. Z. Ahmed et al.	2018	Introduces TRACER for repairing compilation errors in student programs using PL techniques and deep learning.	Supports automatic correction for learners.
4	MACER: A Modular Framework for Accelerated Compilation Error Repair	2020	Presents a modular framework for automatically repairing compilation errors, particularly for novices.	Relevant to the automated repair architecture.
5	Mining on Students' Execution Logs and Repairing Compilation Errors Based on Deep Learning — R. Shi et al.	2023	Uses student execution logs and deep learning to generate and repair compilation-error examples.	Relevant to student-oriented correction and training data.
6	What Would Other Programmers Do? Suggesting Solutions to Error Messages — B. Hartmann et al.	2010	Introduces HelpMeOut, recommending fixes based on prior programmers' solutions and source-code context.	Relevant to correction suggestions and contextual fixes.
7	Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research — B. A. Becker et al.	2019	Reviews research on compiler error messages and identifies problems faced by novices.	Provides the research foundation / problem justification.
8	Effective Compiler Error Message Enhancement for Novice Programming Students	2016	Studies enhanced Java compiler error messages and their effect on novice programmers.	Supports human-readable error explanations.
9	On Compiler Error Messages: What They Say and What They Mean — Traver	2010	Examines the information contained in compiler messages and approaches to improve interpretation.	Relevant to understanding and improving diagnostics.
10	Not the Silver Bullet: LLM-enhanced Programming Error Messages are Ineffective in Practice — E. A. Santos, B. A. Becker	2024	Evaluates LLM-enhanced error messages and highlights limitations of relying solely on LLMs.	Important for reliability/validation of LexiGuide.
11	An Empirical Evaluation of the Formative Feedback Supported by Dashboard in the Context of Compilation Error	2023	Examines automated feedback and the challenge of task-specific suggestions for compilation errors.	Supports context-specific feedback.
12	The Effect of Automated Error Message Feedback on Undergraduate Physics Students Learning Python	2022	Studies automated error-message feedback and its effects on students learning programming.	Relevant to the educational application of LexiGuide.

IV. RELEVANCE TO LEXIGUIDE

The surveyed literature provides a foundation for using intelligent models to explain compiler errors. LexiGuide extends this foundation by focusing specifically on lexical errors and by integrating context-aware detection, explanation, and correction into a single system, rather than treating these as separate concerns.

V. RESEARCH GAP AND NOVELTY

The existing research predominantly addresses the use of Large Language Models to explain and resolve general compiler errors. However, there is limited focus on a dedicated system that specifically handles lexical errors by combining context-aware detection, understandable explanation, and correction within a single workflow. LexiGuide addresses this gap by developing an integrated system that analyzes the lexical context of source code, identifies lexical errors, explains their causes in simple language, and provides context-sensitive correction suggestions.

Assessed Novelty: **9.5 / 10**

VI. PROPOSED WORKFLOW / ARCHITECTURE

The proposed system follows a sequential pipeline that takes raw source code as input and produces corrected code or actionable guidance as output, as illustrated below.

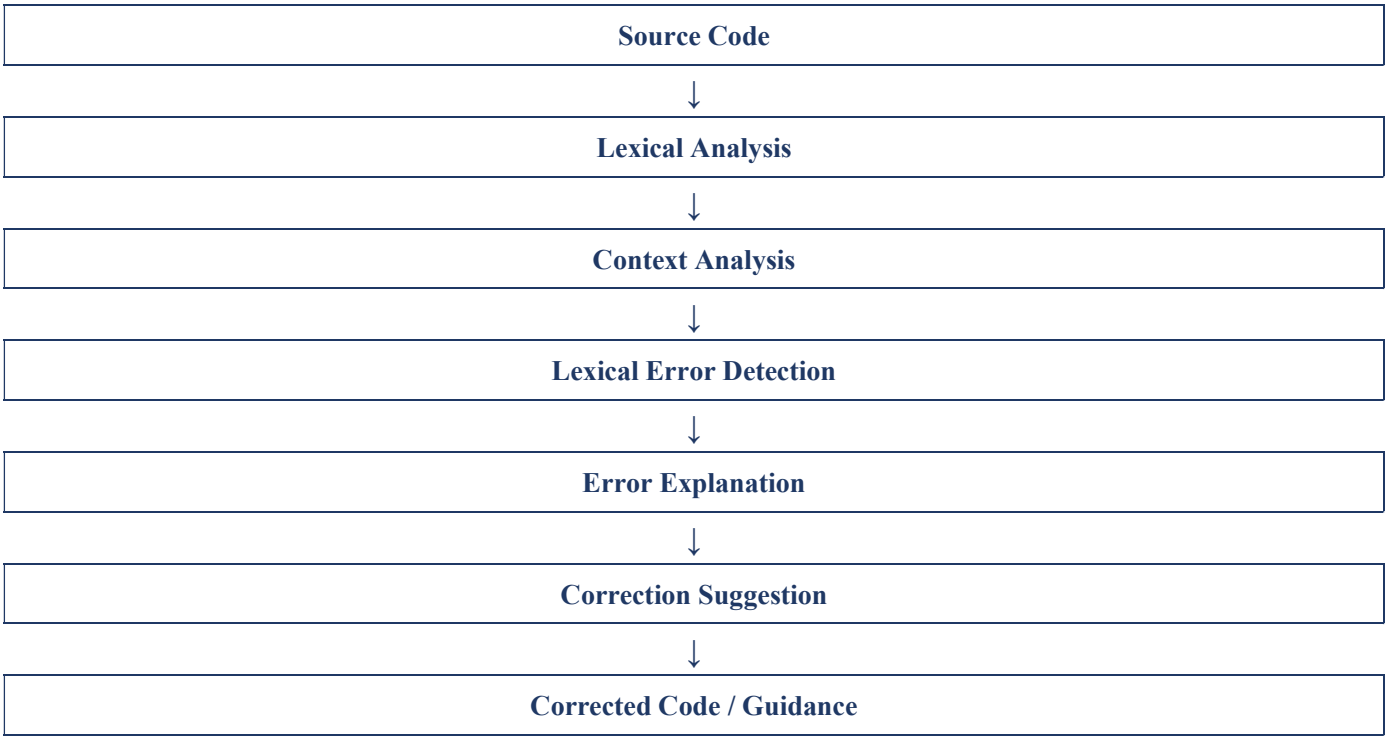


Fig. 1. Proposed LexiGuide processing pipeline.

VII. CONCLUSION

LexiGuide aims to improve the way lexical errors are handled by combining context-aware detection, clear explanations, and automatic correction suggestions within a single system. By addressing the limitations of traditional compiler error messages, the proposed system can make error resolution faster, easier to understand, and

more useful for beginners. The literature survey indicates a clear opportunity to develop a focused solution for lexical errors that integrates these capabilities into one unified framework. Future work will focus on implementing and empirically evaluating the proposed pipeline against existing compiler diagnostics.

VIII. REFERENCES

[1] P. Widjojo and C. Treude, "Addressing Compiler Errors: Stack Overflow or Large Language Models?" arXiv preprint, 2023.

[2] X. Li et al., "TransRepair: Context-aware Program Repair for Compilation Errors," Elsevier, 2022.

[3] U. Z. Ahmed et al., "Compilation Error Repair: For the Student Programs, From the Student Programs," Microsoft Research, 2018.

[4] "MACER: A Modular Framework for Accelerated Compilation Error Repair," PubMed Central (PMC), 2020.

[5] R. Shi et al., "Mining on Students' Execution Logs and Repairing Compilation Errors Based on Deep Learning," MDPI, 2023.

[6] B. Hartmann et al., "What Would Other Programmers Do? Suggesting Solutions to Error Messages," hci.stanford.edu, 2010.

[7] B. A. Becker et al., "Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research," 2019.

[8] "Effective Compiler Error Message Enhancement for Novice Programming Students," Taylor & Francis Online, 2016.

[9] Traver, "On Compiler Error Messages: What They Say and What They Mean," Wiley Online Library, 2010.

[10] E. A. Santos and B. A. Becker, "Not the Silver Bullet: LLM-enhanced Programming Error Messages are Ineffective in Practice," 2024.

[11] "An Empirical Evaluation of the Formative Feedback Supported by Dashboard in the Context of Compilation Error," Wiley Online Library, 2023.

[12] "The Effect of Automated Error Message Feedback on Undergraduate Physics Students Learning Python," Springer, 2022.

TEAM MEMBERS

- Rupayan Chattaraj (24BCE0716)
- Nagendra (24BC2185)
- Himanshu (24BCE0955)